

FerroScope: Characterizing Hardware–Compiler Interplay in C-to-Rust Migration

Dalton Davis Tapti Palit
University of California, Davis
{drdavis, tpalit}@ucdavis.edu

Abstract

Migrating C code to Rust can significantly improve software security. While Rust is often claimed to match C’s performance, few systematic studies examine this claim in depth. Migration strategies range from naive, line-by-line conversions to sophisticated idiomatic rewrites, introducing tradeoffs in correctness and performance. However, a systematic understanding of why migrated Rust code incurs performance overhead remains lacking. To address this, we present FerroScope, an in-depth evaluation characterizing the performance of C and migrated Rust. Our study reveals that Rust’s safety checks do not inherently dominate runtime overhead, as compiler and microarchitectural optimizations often mask their costs. Across 8 of the 12 evaluated benchmarks, at least one Rust implementation achieved performance within a 10% margin of the corresponding C baseline or exceeded it. Our study provides important insights for future work on efficient and secure code migration.

1. Introduction

Rust [10] is a memory-safe, systems programming language that is increasingly gaining popularity. Unlike other memory-safe programming languages such as Java [61] and Go [75], Rust does not rely on garbage collection [11] to ensure temporal safety. Instead, the Rust compiler ensures that the Rust code is free from temporal safety vulnerabilities at compile-time. Performing these checks at compile time allows Rust to mitigate the performance overhead typically associated with temporal memory safety. Due to Rust’s memory safety and performance characteristics, system software such as operating systems [18, 59, 66, 77] and device drivers [25, 26] are being increasingly written in Rust. Recently, the Linux kernel has officially added support for Rust code [4, 64, 83]. Similarly, parts of web browsers [9] and web servers [31] are being rewritten in Rust.

However, not all of Rust’s safety checks happen at compile time. To guarantee spatial safety, Rust abstractions, such as *vectors* and *strings*, perform runtime checks to detect out-of-bounds memory accesses. Rust also supports runtime-checked pointers, including reference-counted pointers (RC), and types that support interior mutability safely (RefCells). The performance overhead for the runtime safety checks in Safe Rust code stems from many sources—some obvious, and some not so obvious. The obvious source of performance overhead is the cost of executing the

additional *machine instructions* for the check itself. But there exist other non-obvious overhead sources, too. For example, inserting the runtime checks can potentially disrupt compiler optimizations. Bounds checks are essentially assembly-level *branch instructions* and inserting these branches can change how the compiler optimizes the surrounding code.

Rust’s safety checks also impact the hardware microarchitectural characteristics. The increase in the number of branch instructions resulting from the memory safety checks can put pressure on the hardware branch predictor. This could potentially increase the percentage of *branch misses* and stall the CPU pipeline, increasing the runtime overhead. However, on the other hand, if Rust’s memory-safety checks result in *predictable* branch outcomes and branch targets, then the hardware branch predictor can mitigate much of the associated overhead. Similarly, if the memory safety checks involve loading any data from the DRAM (for example, the bounds information), the performance overhead would be masked if hardware caches can capture this data, thus reducing the data access latency.

Furthermore, Rust’s ownership model can also *improve* compiler optimizations in ways that partially offset these costs. The borrow checker’s guarantee that no mutable reference coexists with any other reference to the same memory allows the compiler to enable more aggressive alias analysis and optimization [82].

Clearly, these factors complicate the estimation of the performance of the Rust version of a codebase, given its C counterpart. This presents a challenge for software developers who are considering whether they should rewrite their legacy C codebases to Rust. While it is clear that Rust would outperform other garbage-collected languages, there is less consensus surrounding its performance in comparison to C code. Performance benchmarks for Rust ports remain nuanced and vary by project. For instance, while the Rust implementation of *zlib* [29] reportedly *outperforms* its C baseline, the Rust port of GNU CoreUtils [67] initially *underperformed*. Only after subsequent targeted optimizations, core utilities such as *wc*, and *ls* match or exceed the performance of their original C counterparts [49]. This lack of consensus also prevails in research works measuring C and Rust performance. Some previous works [27, 88] have found that the memory-safety checks contribute to significant overheads, reaching up to $2.49\times$ overhead. On the other hand, other works [63] have found that Rust code can outperform C code for some libraries. More recent comparisons [57, 85] show that well-optimized Rust

implementations typically stay within 5–10% of C/C++ performance on average, with wins in some workloads and small losses in others. However, the reasons for this behavior have not yet been analyzed systematically, and FerroScope aims to fill this gap.

Confounding matters further, different rewriting strategies can have different performance characteristics—from line-by-line C-to-Rust rewrites to a complete redesign and re-implementation of the program functionalities in Rust. Indeed, previous studies [52] have shown that developers translating from C to Rust often begin with a direct, line-by-line conversion to ensure correctness and then refactor toward idiomatic Rust once they are confident of correctness. These performance concerns are not limited to manual transpilation. They have significant consequences for automated transpilers as well, ranging from syntax-directed approaches [34, 36, 87] to LLM-based techniques [37, 89].

We introduce FerroScope, a workload characterization study that decomposes language-level safety costs into compiler and microarchitectural factors. FerroScope contains 12 micro- and macro-benchmarks to conduct the performance comparison. This dataset includes the original C and Rust reimplementations of the benchmarks. In cases where applicable, we provide alternate Rust implementations to explore the impact of the implementation-style on the performance. Each benchmark isolates a specific factor influencing runtime performance. In the microbenchmark category, we focus on searching and sorting algorithms. These algorithms operate on dynamically-sized buffers of data, abstracted as vectors, and therefore, involve many dynamic bounds checks. In the macro-benchmarks category, we include *Skip List* [80] and *PageRank* [62] implementations, and the *LibCsv* [2] and *u8c* [30] libraries.

FerroScope’s performance analysis shows that contrary to common belief, Rust’s safety mechanisms do not *always* dominate runtime overheads. Instead, modern compilers and microarchitectural optimizations often hide these costs entirely. Across 8 of the 12 evaluated benchmarks, at least one Rust implementation achieved performance within a 10% margin of the corresponding C baseline or exceeded it. Our results reveal a three-tier interaction model between language safety mechanisms, compiler optimizations, and microarchitectural optimizations, which explains why Rust’s runtime overheads are often smaller than expected. We further show that, in many cases, the success of the microarchitectural optimizations depends neither on how many memory safety checks are added, nor on the class of algorithm. For example, the iterator-based Rust ports of Merge Sort and Quicksort have very different runtime performance, with Merge Sort being 15% *faster* and Quicksort being 71% *slower* than the C baseline on the AMD machine due to an increased data cache miss rate. Our analysis also uncovered that the compiler sometimes optimizes the Rust code differently from C, resulting in unexpected performance slowdowns.

In summary, our contributions are as follows—

- We present FerroScope, a novel workload characterization that systematically decomposes and analyzes Rust’s language-level safety costs into compiler-level and hardware-level components. FerroScope provides a structured investigation of how Rust’s safety mechanisms interact with compiler optimizations and microarchitectural behavior, providing a foundation for fine-grained performance analysis in C-to-Rust translation.
- We develop a benchmark dataset comprising 12 micro- and macro- benchmarks, including both original C and reimplemented Rust versions. The suite encompasses a diverse set of workloads, including Skip List, LibCsv, and u8c, facilitating broad empirical evaluation.
- Through extensive measurement, we demonstrate that Rust’s safety mechanisms do not necessarily dominate runtime overhead. We show that the hardware branch predictor and caches often absorb the branch check costs. We further show that no single implementation strategy (for example, index-based vs. iterator-based) consistently outperforms the other, and that compiler mis-optimizations can introduce overhead unrelated to safety checks in both Rust and C.

2. Background

2.1. Rust Memory Safety

Compile-time Checks. Rust achieves memory safety through a combination of compile-time and runtime checks. Rust enforces a strict ownership system at compile time, encompassing three core concepts—*Ownership*, *Lifetime*, and *Borrowing*. In Rust, every object has a single owner, and when that owner goes out of scope, the object is automatically dropped and its memory freed. References can borrow an object’s value, but the compiler ensures these references never outlive the owner. Additionally, the Rust Borrow Checker also enforces the *Mutability-xor-aliasing* rule, which ensures that heap variables can have either multiple immutable references or a single mutable reference, but never both simultaneously. Together, these rules provide strong temporal safety guarantees against *Use-After-Free* and *Double-Free* bugs, and prevent data races.

Runtime Checks. Rust enforces spatial safety through runtime bounds checking. Every time a vector or slice element is accessed by index, Rust inserts a check to verify that this index is within the object range. These checks eliminate spatial safety bugs but can incur some performance overhead. Consider the simplified memory safety check for a slice indexing operation shown in Figure 1. The index operation is performed by the `core::slice::index::SliceIndex<[T]>::index` function, in the Rust standard library. As the assembly instructions show, this check involves (i) loading the slice’s *size* into the `rdi` register, (ii) performing a test operation, and finally (iii) performing a conditional branch operation using the `jne` instruction. As we will discuss next, these instructions interact with compiler and microarchitectural optimizations, thus impacting runtime performance.

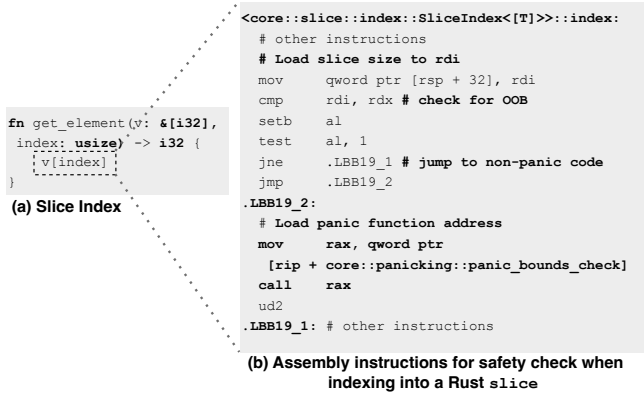


Figure 1: Simplified Rust Runtime Safety Check.

2.2. Compiler Toolchains and Optimizations

The Rust compiler (rustc) uses a multi-stage compilation pipeline, which lowers the Rust code to assembly code. Both rustc and clang use the *same* LLVM backend for low-level optimizations. Thus, a significant part of the code generation is similar across Rust and C.

Multiple compiler optimizations can impact the runtime performance of Rust’s safety checks. For example, for the slice index check code provided in Figure 1, one of the simplest compiler optimizations is to inline the index function. We observed that for all optimization levels higher than `-O0`, the compiler consistently inlined these safety checks. Then, suppose the code contains multiple index operations into the same slice. In that case, the compiler might optimize loading the slice size from memory by promoting the bounds value to a register and retaining it between subsequent accesses, thus amortizing the slow memory access. And finally, the compiler can choose to rewrite the two-instruction sequence of the conditional branch (`jne`) followed by the memory load (`mov rax, [panic_bounds_check]`) into a single conditional memory load (`cmov`) instruction, thus reducing the instruction count. We discuss potential impacts of this optimization in §5.4.

2.3. Microarchitectural Optimizations

Compilers optimize binaries to exploit microarchitectural structures such as caches, branch predictors, TLBs, and hardware prefetchers. However, language-level safety guarantees can disrupt the application of such optimizations. Specifically, the additional memory loads introduced by safety checks (e.g., bounds checks in Figure 1) can be absorbed if they hit in the L1 or L2 data caches, but can result in performance penalties if they evict frequently accessed data or increase instruction cache pressure. Similarly, conditional branches in bounds checks are highly predictable when executed repeatedly with the same outcome, allowing the hardware branch predictor to almost entirely absorb their cost. Hardware prefetchers and TLBs further mitigate latency by anticipating future accesses and caching virtual-to-physical translations [44, 68], respectively, though mispredictions or cache thrashing can negate these benefits.

TABLE 1: Microbenchmark Characteristics.

Microbenchmark	Data Representation	Input Size
Breadth-First Search	Adjacency Matrix	30,000 nodes
Depth-First Search	Adjacency Matrix	30,000 nodes
Single-Source Shortest Path	Struct + Adjacency List	30,000 nodes
Binary Search	Sorted Integer Array	20,000,000 elements
Merge Sort	Integer Array	100,000,000 elements
Quicksort	Integer Array	100,000,000 elements
Selection Sort	Integer Array	200,000 elements
Matrix Multiplication	2D Integer Matrix	2000 × 2000 matrix

3. FerroScope Benchmarks

FerroScope consists of micro-benchmarks, application and library-level macrobenchmarks (Table 1 and Table 2). Our benchmark selection is motivated by three goals: (1) using workloads that are commonly used in systems-level code, or used in prior C-to-Rust transpilation studies [36, 60, 87, 88], (2) achieving an overall balance between different access patterns (regular, semi-regular, and irregular), and (3) stress-testing Rust’s runtime checks.

Our benchmarks include sorting, searching, graph traversal, and matrix multiplication algorithms with diverse access patterns [5, 7, 12, 17, 21, 23, 24, 48, 70, 72, 79]. To evaluate irregular, pointer-chasing access patterns, we include Skip Lists [45, 65, 74, 80] and PageRank [17, 62, 70]. These represent the linked and graph structures widely used in production systems like LevelDB and Redis. We evaluate PageRank on the LiveJournal social network graph [15, 50], which contains ~ 4 M nodes and ~ 35 M edges.

For control-intensive workloads with regular streaming access patterns, we utilize two library benchmarks. The first is U8C [30], a UTF-8 library used in Rust crates such as `serde` [69] and `ripgrep` [19]. We benchmark the `u8next` decoder, a DFA-based state machine with multiple branches and state transitions, on a text file containing ~ 2.8 million characters [22]. The second is LibCSV [43], an ANSI-C CSV parsing library. Here, we benchmark the `csv_parse` function, which relies on a byte-oriented state machine with branch-heavy control flow, using the COVID-19 Open Research Dataset [13, 84] (~ 1 million rows, 1.65 GB). Together, these benchmarks span the broad spectrum of access patterns and computational characteristics relevant to systems software ported to Rust.

4. Methods and Setup

4.1. Evaluation Methodology

As discussed in §2.2 and §2.3, the microarchitectural characteristics of a workload depend on the program-level data access patterns and also how the compiler optimizes the program. To analyze the workload’s microarchitectural characteristics, FerroScope uses Pin tools [8] to record the dynamic instruction composition and the Hardware Performance Counters, discussed in §2.3, to record the microarchitectural characteristics of the workload. In particular, FerroScope records the following components—

- The L1 and L2 data cache misses, which allow us to reason about (i) the cache locality of the memory safety

TABLE 2: Application and Library Characteristics (N: Total Nodes, E: Total Edges)

Benchmark	Type	Operation	Operation Breakdown	Access Pattern	Data Representation	Input Size
Skip List	Application	3N/2 Inserts, 3N/4 Removes, N Searches	N inserts, N/2 updates; N/2 hit, N/4 miss removes; N/2 hit, N/2 miss searches	Irregular	Linked Node	N = 5M
PageRank	Application	Graph Traversal	Sparse Graph Processing	Irregular	Adjacency List	N ~ 4M, E ~ 35M
U8C	Library	UTF-8 Decode	DFA-based State Machine	Regular	Byte Array	~ 2.8M chars
LibCSV	Library	CSV Validation	Single-pass Streaming Parser / FSM	Regular	Byte Array	~ 1.6B chars

metadata accessed as part of the safety checks, and (ii) any alteration to the compiler’s memory access optimizations due to Rust’s abstractions and safety-checks.

- The hardware Branch Predictor misses, which allow us to isolate any performance overhead due to branch instructions. A higher number of branch misses indicates that either (i) the Rust’s safety checks inserted irregular branches that the branch predictor could not predict successfully, or (ii) Rust’s abstractions and safety checks caused an increase in the Branch Predictor pressure.
- The dynamic instruction mix of the workload using a Pin tool [8]. We specifically focus on the memory load and store operations, and the conditional branch instructions because they can potentially stall the CPU pipeline.

4.2. Evaluation Setup

Compilers. All C benchmarks were compiled with the clang-21.1.8 compiler, and the Rust benchmarks were compiled using rustc-1.94.1. Both clang-21.1.8 and rustc-1.94.1 use the LLVM-21 backend to compile to x86_64 assembly code. This ensures that both the C and Rust benchmarks are optimized by the same LLVM optimization pipeline. To isolate the impact of Rust’s memory safety checks, we use a modified version of Rust that disables memory-safety related runtime checks [88]. We call this compiler the Rust- compiler. We ported these compiler modifications to the latest stable Rust compiler (version 1.94.1). In particular, the following safety checks were disabled in this compiler version—bounds checking for arrays, vectors, slices, and string/byte slices, and integer overflow detection, which is disabled in the base Rust version as well. All other checks, such as reference counting, remain intact. All benchmarks were evaluated at the -O3 optimization mode, using Release Mode builds.

Perf and Pin tools. To characterize microarchitectural behavior, we use hardware performance counters [3, 28, 76] to collect branch mispredictions, L1-D cache misses, and L2 cache misses and instruction-level instrumentation with Intel Pin [8, 55] to obtain dynamic traces of all executed branches, loads, and stores. We do not report L3 misses because the L3 cache is shared across cores and can be interrupted by OS activity [40]. To the best of our

knowledge per-process or per-core metrics are not provided by either vendor. We focus our analysis on cache and branch-related behavior, as they are the primary microarchitectural components that account for the performance differences observed between Rust and C. All measurements excluded any setup or data initialization phase.

Hardware Details. We conducted our experiments on the CloudLab [33] c6525-25g nodes in the Utah cluster for AMD and sm110p nodes in the Wisconsin cluster for Intel. Both systems used Linux v5.15.0-168-generic. The AMD node is a Dell PowerEdge C6525 server equipped with an AMD EPYC 7302P processor. The machine had 16 physical cores (32 threads with SMT) running at 3.0 GHz and a 155 W TDP. Each core has 32 KiB L1D and 32 KiB L1I caches, 512 KiB L2 cache, and 16 MiB L3 per CCX (128 MiB total). The Intel node is a Supermicro SuperServer 110P equipped with an Intel Xeon Silver 4314 processor (Ice Lake-SP, Sunny Cove microarchitecture) [47]. The machine had 16 physical cores (32 threads with Hyper-Threading) with a 2.4 GHz base clock (3.4 GHz turbo) and a 135 W TDP. Each core has 48 KiB L1D and 32 KiB L1I caches, 1.25 MiB L2 cache, and a non-inclusive shared 24 MiB L3 cache [38].

5. Benchmark Results Analysis

5.1. Performance Equivalence of Rust and C

Table 3 and Table 4 present the runtime overhead of the benchmarks for AMD and Intel machines, respectively. To minimize variance, all reported results are averaged across three independent runs. Our first observation is that despite the Rust code performing additional memory-safety checks, for 8 out of 12 benchmarks, at least one implementation variant incurs less than 10% overhead over the C implementation. Of these 8 variants, 5 outperform the C implementation on both architectures. Moreover, for Rust, the iterator-based Matrix Multiplication and Merge Sort implementations, the UTF-8 Decode (u8next) benchmark, the LibCsv, and the Skip List benchmarks *outperformed* their C counterparts. Even among the index-based implementations that use random access into vectors—a pattern that necessarily incurs bounds checks on every

access—several benchmarks exhibit modest overhead. This demonstrates that bounds checks alone do not necessarily lead to significant performance degradation.

Safety Check Overhead. We observe that runtime bounds checks contribute to performance overhead for several, but not all, benchmarks. By using the Rust- compiler, we isolate the effect of runtime checks on performance. Eliminating bounds checks substantially reduces overhead for Matrix Multiplication (from $1.86\times$ to $1.67\times$ on AMD, $1.71\times$ to $1.40\times$ on Intel), recursive Merge Sort with indexing (from $1.68\times$ to $1.23\times$ on AMD, $1.58\times$ to $1.15\times$ on Intel), and PageRank (from $1.40\times$ to $1.24\times$ on AMD, $1.43\times$ to $1.20\times$ on Intel), but does not account for all of it. However, for Selection Sort and iterative Merge Sort, bounds check elimination removes the overhead entirely. Selection Sort drops from $2.93\times$ to $0.99\times$ on AMD and from $1.57\times$ to $0.83\times$ on Intel, while iterative Merge Sort drops from $1.44\times$ to $0.91\times$ on AMD and from $1.43\times$ to $0.94\times$ on Intel. In both of these cases, performance matches or outperforms C. In contrast, bounds check elimination makes less than a 10% difference on BFS, DFS, and Binary Search, despite their presence (as suggested by the decrease in conditional branches ranging from 30% to 60% comparing to Rust-). In these cases, Rust’s performance overhead originates elsewhere. We investigate these overhead sources in §5.3.

Overhead Sources Beyond Safety Checks. Moreover, out of the total 19 benchmarks and variants in our dataset, even the Rust- version underperformed the baseline C version for 5 (AMD) and 6 (Intel) benchmarks, respectively. Out of these BFS, Binary Search, and the Quicksort iterator-based variant exhibited more than 20% overhead than the C version. This shows that there exists overhead sources in addition to Rust’s runtime memory safety checks. Previous work by Zhang et al. [88] attributed any remaining performance overhead after disabling the memory-safety checks to ASCII to UTF-8 conversion for string-based workloads. However, our Rust benchmarks do not perform any additional ASCII to UTF-8 conversion over the C baseline. As we will discuss in detail in §5.3 and §5.4, the additional overhead occurs due to a complex interplay between compiler optimizations and microarchitectural components. Thus, the performance of Rust is not just the cost of the safety checks over the C baseline runtime.

5.2. Implementation Strategy Impact

We observed that the implementation strategy played a significant role in the Rust overhead, and also that in terms of performance, no particular strategy consistently outperformed others.

Sorting Algorithm Variants. The Merge Sort algorithm has two algorithmic variants—a top-down recursive (TDR), and a bottom-up iterative (BUI) variant. The top-down recursive (TDR) variant follows the classic divide-and-conquer approach, recursively splitting the array in half until reaching single-element subarrays, then merging upward. Each call to merge allocates and frees two temporary arrays for the left and right halves. The bottom-up iterative (BUI)

variant instead uses a single pre-allocated temporary buffer and iteratively merges subarrays of increasing width (1, 2, 4, ...), copying the full result back after each pass. Because the bottom-up variant skips making additional memory copies, one would expect that it outperforms the top-down variant. We indeed observe this effect, with the C top-down benchmark taking ~ 12 seconds to sort a 100-million sized integer array, while the bottom-up benchmark takes ~ 9.5 seconds on both AMD and Intel architectures. However, one might expect that the overhead of the Rust translations of both these algorithm variants would remain consistent. However, as shown in Table 3 and Table 4, Rust has a 68% overhead for the top-down algorithm, but a 44% overhead for the bottom-up algorithm on the AMD, and 58% and 43%, for Intel, respectively. Thus, not only is the top-down variant *slower for the baseline C*, it has a *higher overhead when transpiled to Rust*. We discuss potential reasons for this behavior using microarchitectural metrics in §5.3.1.

Additionally, it is possible to implement the same algorithm in both imperative and functional programming styles. For example, the sort algorithms can be implemented in an imperative-style, using indexing operations into the vec objects, or using a more functional programming style, using iterators and map and fold operations. One might expect that because the iterator-based implementations do not involve runtime bounds checks, they would *always* outperform the indexing-based implementations. However, as shown in Table 3 and Table 4 we observed that this was true only for Matrix Multiplication and Merge Sort, where the iterator-style implementations matched or *improved* the Rust runtime over C. For Quicksort, the iterator-based implementation encountered a 64% and 71% overhead, while the indexing-based implementation suffered only a 7% and 2% overhead for Intel and AMD, respectively, discussed in §5.3.1. This shows that despite programmer intuition, there is a lack of a definitive superior implementation strategy.

5.3. Hiding Rust Overheads

We now discuss the benchmark overheads using the microarchitectural characteristics and dynamic instruction traces. For all but 4 benchmarks, at least one Rust implementation had comparable (within 10%) performance or *outperformed* C. Benchmarks involving few bounds checks, such as graph traversals, are natural candidates for low overhead. Indeed, BFS incurs only 10% overhead on AMD (23% on Intel), and DFS only 5% on both architectures. However, we observe that surprisingly, even for some benchmarks that contain a high number of bounds checks in tight loops (SSSP, DFS, and Quicksort), the Rust overhead is less than 10%. Below, we discuss the microarchitectural components that hide Rust’s overheads (Table 3 and Table 4).

5.3.1. Impact of Branch Predictors. Indexing into a vec or a slice invokes a bounds-check consisting of three operations—i) loading the size of the vec or slice object, ii) comparing the index with the size, and finally iii) depending on the comparison result retrieving the element at the index or panicking. Thus, if the vector’s size is not promoted to a

Benchmark	Time		Rust										Rust--									
	Rust	Rust--	IPC	Cycles	Total Inst.	Cond Branches	Stores	Loads	Branch Misses	L1D Misses	L2D Misses	IPC	Cycles	Inst.	Cond Branches	Stores	Loads	Branch Misses	L1D Misses	L2D Misses		
BFS	1.10×	1.01×	1.37×	1.22×	1.67×	1.40×	2.51×	3.00×	1.00×	1.01×	1.01×	1.04×	1.12×	1.17×	1.00×	2.51×	2.33×	1.00×	1.02×	1.01×		
Binary Search	1.26×	1.25×	1.16×	1.27×	1.48×	1.04×	1.11×	1.02×	0.06×	0.27×	0.27×	1.07×	1.26×	1.35×	0.74×	1.11×	1.02×	0.06×	0.27×	0.27×		
DFS	1.05×	0.97×	1.60×	0.98×	1.57×	1.60×	1.43×	1.40×	1.00×	0.63×	0.95×	1.03×	0.97×	1.00×	1.00×	1.29×	1.00×	1.00×	0.63×	0.97×		
Matrix Mult	1.86×	1.67×	1.75×	1.84×	3.23×	9.97×	1.00×	3.00×	1.00×	1.22×	1.20×	0.74×	1.66×	1.23×	1.00×	1.00×	2.00×	1.00×	1.21×	1.13×		
MM Iterator	0.95×	0.95×	1.06×	0.95×	1.00×	1.01×	0.67×	1.00×	1.00×	1.01×	1.03×	1.06×	0.94×	1.00×	1.00×	0.67×	1.00×	1.00×	1.01×	1.03×		
Merge Sort TDR	1.68×	1.23×	0.70×	1.69×	1.18×	2.13×	1.23×	1.39×	18.60×	1.21×	1.16×	1.03×	1.23×	1.27×	1.62×	1.25×	1.56×	1.01×	1.17×	2.09×		
Merge Sort BUI	1.44×	0.91×	0.62×	1.51×	0.93×	2.69×	1.10×	1.00×	20.52×	1.00×	1.00×	1.13×	0.88×	0.99×	0.98×	1.09×	1.46×	1.00×	1.00×	0.95×		
Merge Sort Iterator	0.85×	0.84×	1.36×	0.80×	1.09×	1.53×	1.12×	1.27×	0.09×	1.04×	2.02×	1.37×	0.79×	1.09×	1.53×	1.12×	1.27×	0.09×	1.04×	2.00×		
Quicksort	1.02×	0.96×	1.29×	1.00×	1.29×	1.63×	1.04×	1.02×	0.98×	1.00×	0.95×	0.98×	0.96×	0.94×	1.01×	0.97×	0.98×	0.99×	1.00×	1.00×		
Quicksort Iterator	1.71×	1.65×	2.25×	1.47×	3.31×	3.11×	2.79×	3.42×	1.04×	3.38×	4.61×	1.98×	1.53×	3.02×	2.36×	3.22×	3.71×	1.05×	3.38×	4.65×		
Selection Sort	2.93×	0.99×	0.75×	2.90×	2.18×	2.40×	1.00×	1.00×	0.94×	1.00×	0.62×	0.82×	1.00×	0.82×	1.00×	1.00×	1.00×	0.99×	1.00×	1.01×		
SSSP	0.56×	0.54×	1.70×	0.55×	0.94×	2.50×	0.00×	1.00×	152.74×	1.76×	2.00×	1.17×	0.53×	0.63×	1.25×	0.00×	1.00×	137.67×	1.82×	2.01×		
SSSP Iterator	0.56×	0.54×	1.70×	0.55×	0.94×	2.50×	0.00×	1.00×	140.56×	1.76×	1.99×	1.28×	0.54×	0.69×	1.50×	0.00×	1.00×	289.83×	1.81×	1.97×		
CSV Valid	0.87×	0.92×	0.69×	1.06×	0.73×	0.91×	0.66×	0.67×	1.00×	0.08×	0.74×	0.76×	0.95×	0.73×	0.91×	0.66×	0.67×	1.00×	0.79×	0.65×		
PageRank	1.40×	1.24×	1.07×	1.40×	1.50×	2.10×	1.05×	2.01×	1.00×	1.11×	1.05×	0.78×	1.24×	0.96×	1.02×	1.04×	1.49×	0.99×	1.10×	1.04×		
Skiplist Insert	0.75×	0.73×	3.38×	0.74×	2.51×	3.37×	3.80×	2.46×	1.00×	0.72×	0.60×	3.18×	0.73×	2.31×	2.86×	3.71×	2.14×	1.00×	0.71×	0.62×		
Skiplist Remove	0.69×	0.69×	3.70×	0.66×	2.45×	3.51×	4.58×	2.06×	0.98×	0.79×	0.53×	3.34×	0.66×	2.21×	3.03×	4.42×	1.92×	0.96×	0.79×	0.54×		
Skiplist Search	0.68×	0.68×	2.72×	0.68×	1.86×	2.71×	6.92×	1.78×	0.99×	0.64×	0.56×	2.54×	0.68×	1.73×	2.35×	6.92×	1.64×	0.98×	0.63×	0.56×		
u8next	0.67×	0.76×	1.57×	0.70×	1.09×	1.12×	1.00×	1.00×	0.91×	0.79×	0.04×	1.43×	0.77×	1.09×	1.12×	1.00×	1.00×	0.86×	0.79×	0.04×		

TABLE 3: Rust/C performance ratios (AMD). (Time Column: Values $\leq 1.00\times$ are in green and values $\geq 1.10\times$ are in red)

Benchmark	Time		Rust										Rust--									
	Rust	Rust--	IPC	Cycles	Total Inst.	Cond Branches	Stores	Loads	Branch Misses	L1D Misses	L2D Misses	IPC	Cycles	Inst.	Cond Branches	Stores	Loads	Branch Misses	L1D Misses	L2D Misses		
BFS	1.23×	1.22×	1.40×	1.19×	1.67×	1.40×	2.73×	3.00×	1.00×	1.24×	0.81×	0.96×	1.22×	1.17×	1.00×	2.73×	2.33×	1.00×	1.17×	0.80×		
Binary Search	1.15×	1.13×	1.27×	1.16×	1.48×	1.04×	1.11×	1.02×	0.06×	0.35×	0.28×	1.20×	1.13×	1.35×	0.74×	1.11×	1.02×	0.06×	0.35×	0.28×		
DFS	1.05×	1.00×	1.41×	1.12×	1.57×	1.60×	1.43×	1.40×	0.99×	0.51×	0.76×	1.00×	1.00×	1.00×	1.00×	1.29×	1.00×	0.99×	0.52×	0.92×		
Matrix Mult	1.71×	1.40×	2.02×	1.60×	3.23×	9.97×	1.00×	3.00×	1.00×	1.07×	1.01×	0.91×	1.35×	1.23×	1.00×	1.00×	2.00×	1.00×	1.01×	1.01×		
MM Iterator	1.01×	1.02×	0.99×	1.01×	1.00×	1.01×	0.67×	1.00×	1.00×	1.96×	1.53×	1.00×	1.00×	1.00×	1.00×	0.67×	1.00×	1.00×	2.13×	1.59×		
Merge Sort TDR	1.58×	1.15×	0.79×	1.49×	1.18×	2.13×	1.46×	1.26×	18.19×	1.07×	1.72×	1.11×	1.14×	1.26×	1.61×	1.76×	1.60×	0.98×	0.96×	0.80×		
Merge Sort BUI	1.43×	0.94×	0.66×	1.41×	0.93×	2.69×	1.10×	1.00×	20.99×	0.94×	1.46×	1.08×	0.92×	0.99×	0.98×	1.09×	1.46×	1.00×	0.95×	0.97×		
Merge Sort Iterator	0.66×	0.65×	1.72×	0.63×	1.08×	1.53×	1.65×	1.34×	0.05×	0.97×	0.75×	1.68×	0.64×	1.08×	1.53×	1.65×	1.34×	0.05×	0.97×	0.76×		
Quicksort	1.07×	0.99×	1.21×	1.07×	1.29×	1.63×	1.04×	1.02×	1.03×	0.59×	0.87×	0.95×	0.99×	0.94×	1.01×	0.97×	0.98×	0.99×	1.23×	1.01×		
Quicksort Iterator	1.64×	1.67×	2.11×	1.57×	3.31×	3.11×	2.79×	3.42×	1.10×	48.32×	2.37×	1.91×	1.58×	3.02×	2.36×	3.22×	3.71×	1.11×	48.83×	2.57×		
Selection Sort	1.57×	0.83×	1.36×	1.60×	2.18×	2.40×	1.00×	1.00×	1.00×	3.03×	2.97×	0.98×	0.84×	0.82×	1.00×	1.00×	1.00×	1.03×	2.82×	32.70×		
SSSP	0.78×	0.77×	1.25×	0.75×	0.94×	2.50×	0.00×	1.00×	1758.61×	1.44×	2.25×	0.83×	0.75×	0.63×	1.25×	0.00×	1.00×	1767.93×	0.26×	3.26×		
SSSP Iterator	0.77×	0.78×	1.24×	0.76×	0.94×	2.50×	0.00×	1.00×	1772.72×	1.44×	2.45×	0.91×	0.75×	0.69×	1.50×	0.00×	1.00×	1790.17×	1.50×	2.70×		
CSV Valid	0.96×	0.96×	0.78×	0.93×	0.73×	0.91×	0.66×	0.67×	0.78×	0.13×	0.34×	0.78×	0.93×	0.73×	0.91×	0.66×	0.67×	0.78×	0.19×	0.57×		
PageRank	1.43×	1.20×	1.04×	1.45×	1.50×	2.11×	1.07×	2.01×	0.96×	1.06×	1.17×	0.80×	1.20×	0.96×	1.02×	1.03×	1.49×	0.97×	1.04×	1.11×		
Skiplist Insert	0.96×	0.95×	2.66×	0.94×	2.51×	3.37×	3.80×	2.47×	0.99×	0.63×	0.76×	2.49×	0.93×	2.31×	2.86×	3.71×	2.14×	1.01×	0.64×	0.78×		
Skiplist Remove	0.93×	0.93×	2.64×	0.93×	2.45×	3.51×	4.58×	2.06×	0.98×	0.63×	0.77×	2.40×	0.92×	2.21×	3.03×	4.42×	1.92×	0.99×	0.63×	0.77×		
Skiplist Search	0.93×	0.93×	1.98×	0.94×	1.86×	2.71×	6.92×	1.78×	1.00×	0.64×	0.79×	1.87×	0.92×	1.73×	2.35×	6.92×	1.64×	0.96×	0.65×	0.80×		
u8next	0.52×	0.50×	2.14×	0.51×	1.09×	1.12×	1.00×	1.00×	1.20×	0.03×	0.00×	2.20×	0.50×	1.09×	1.12×	1.00×	1.00×	0.93×	0.03×	0.00×		

TABLE 4: Rust/C performance ratios (INTEL). (Time Column: Values $\leq 1.00\times$ are in green and values $\geq 1.10\times$ are in red)

Benchmark	C	Rust	Rust--
BFS	9	60.0k	60.0k
Binary Search	1	892.9M	892.9M
DFS	0	0	0
Matrix Mult	0	0	0
MM Iterator	4.1k	4.0M	4.0M
Merge Sort TDR	2.53B	1.02B	2.94B
Merge Sort BUI	2.74B	1.00B	2.84B
Merge Sort Iterator	2.53B	3.17B	3.17B
Quicksort	0	0	38.6M
Quicksort Iterator	0	125.0M	125.0M
Selection Sort	0	0	0
SSSP	1.35B	5	5
SSSP Iterator	1.35B	4	4
CSV Valid	12.4M	20.1M	20.1M
PageRank	4	103	0
Skiplist Insert	15.0M	37.5M	37.5M
Skiplist Remove	1	497	497
Skiplist Search	0	5	5
u8next	0	0	0

TABLE 5: CMOV instruction counts for C, Rust, and Rust-- (Counts are identical across AMD and Intel).

register by the Rust compiler and results in a long latency memory load operation to resolve the branch condition, it can stall the CPU pipeline. Therefore, any benchmark involving frequent bounds checks, such as random index-

based sorting benchmarks, could be expected to incur a high overhead. Yet, across the 19 benchmarks and variants, branch predictor misses *decreased* or stayed within 10% of the baseline C version in most cases—15 of 19 on AMD and 13 of 19 on Intel. This indicates that the hardware branch predictor can typically correctly predict the branch targets of the bounds checks branches with very high accuracy.

Cases where BPs Hide the Costs. We observe that for the DFS and index-based Quicksort benchmarks, despite incurring a roughly 60% increase in conditional branches on both architectures, the number of branch misses remains within 3% of the C implementation or less. Both of these cases result in at most a 7% increase in runtime overhead overall. More impressively, the Skip List operations show a drastic increase in branch instructions, ranging from 2.71× for the Search operation to 3.51× for the Remove operation, but with an equal or better branch miss count to the C baseline. Combined with improved cache performance, discussed in §5.3.2, this results in an average 30% runtime improvement on AMD and 6% runtime improvement on Intel, respectively.

Iterators Elide Bounds Checks. Unlike index-based accesses, using iterators to traverse a vec or slice variable, does not involve bounds checks at all. However, our

results show that the use of iterators can *generally* aid in reducing bounds checking overhead. For example, the iterator-based Matrix Multiplication benchmark successfully removed nearly all bounds checks and saw only a 1% increase in the dynamic branch count over C and equal branch misses, resulting in a 5% *runtime improvement* for AMD and a small 1% decrease for Intel. A similar pattern holds for the iterator-based, top-down recursive Merge Sort implementation. While it incurs a modest increase in dynamic memory and branch instructions, it almost completely eliminates branch mispredictions to just 0.09 $\times$ and 0.05 $\times$ for AMD and Intel, respectively, compared to the C baseline. This is a striking difference compared to the $\sim 18\times$ increase in branch misses seen in the index-based Top-Down, Recursive variant. Combined with competitive cache performance, the iterator implementation ultimately *outperforms* C with runtime overheads of 0.85 $\times$ for AMD and 0.66 $\times$ for Intel. We note that compared to the baseline C version, the iterator-based Merge Sort variant *increases* the number of conditional checks to be performed. These checks stem from iterator semantics ensuring that the left, right, and output array iterators are still viable in the Merge function’s hot loop. In contrast, the C code only checks if the left and right array indices are still valid, resulting in 2 instead of 3 checks.

Cases where BPs Partially Hide the Costs. The iterator-based Quicksort also performs worse than the C baseline (71% on AMD, 64% on Intel) despite the branch predictor limiting the branch miss increase to at most 10%, even with a 3.11 $\times$ increase in dynamic branches. In this case, the additional memory operations introduced by the iterators (a 2.79 $\times$ increase in stores and 3.42 $\times$ increase in loads) degrade cache performance (L1D misses increase by 3.38 $\times$ on AMD and 48.32 $\times$ on Intel), outweighing the branch predictor’s effectiveness. We note that the higher dynamic branch count for the iterator-based Quicksort benchmark likely includes sources other than memory-safety checks, since disabling the memory-safety checks using the Rust-compiler does not eliminate these executed branch instructions. This demonstrates that iterators are not universally beneficial. Contrary to what we observe in Merge Sort and Matrix Multiplication, iterators can introduce overhead that exceeds the cost of bounds checks, and the programmer must weigh these tradeoffs on a case-by-case basis.

Cases Where BPs Cannot Hide the Costs. We observed that for benchmarks such as Merge Sort and SSSP, the Rust variants exhibit a *significant increase* in the number of executed branch instructions, resulting in an increased pressure on the branch predictors. Thus, the branch mispredictions are naturally higher for Rust than C. This is particularly evident in the Merge Sort benchmark. Both the top-down recursive (TDR) and bottom-up iterative (BUI) implementations exhibit a substantial increase in dynamic branch instructions for Rust over C: 2.13 $\times$ for TDR and 2.69 $\times$ for BUI on both architectures. This increase corresponds to a pronounced rise in branch mispredictions on both architectures, approximately 18 $\times$ for the top-down

recursive implementation and 20 $\times$ for the bottom-up iterative implementation. We hypothesize that the introduction of additional branches disrupts branch predictor global history. We discuss the compiler behavior responsible for this in §5.4.

5.3.2. Impact of Data Caches. Across both architectures, the Rust implementations of Depth-First Search, SSSP, and Skip List ranges from 21–56% fewer L1D-cache misses compared to the C baseline. On the Intel machine specifically, indexed Quicksort also follows this trend, while Binary Search, LibCSV, and UTF-8 Decode demonstrate even more significant reductions exceeding 50% (up to 97% for UTF-8 Decode). Conversely, on the AMD machine, Breadth-First Search, Quicksort, and Selection Sort also display L1D-cache behavior comparable to their C counterparts. In most cases, these cache-efficiency gains occur despite the Rust implementations issuing more memory instructions overall. This shows that even if the Rust code compiles to more memory operations, potentially due to memory-safety checks, the L1D-cache can capture these accesses, thus limiting their overhead.

To illustrate, the Rust translation of the Skip List search operation issues 1.78 $\times$ more memory load operations and 6.92 $\times$ more store operations than the C baseline. However, the L1D-cache misses *decreases* by 36%, and L2 misses by 44%, resulting in an effective 32% *decrease* in the runtime compared to the baseline C version on the AMD machine. A more modest but similar pattern is found on the Intel machine. Similarly, the Rust version of the Binary Search benchmark, which has only a 11% increase in the dynamic memory store instructions, has a 73% decrease in the L1D-cache miss rate on the AMD machine. The Single-Source Shortest Path benchmark was the only benchmark to exhibit a drastic decrease in dynamic memory instructions accompanied by a high *increase* in cache misses. For SSSP, the L1D-cache misses increase by 76% over the baseline C, but still result in a runtime improvement of 44% on AMD. The behaviors of both the Binary Search and SSSP benchmarks are further influenced by diverging compiler optimization effects, which we explore further in §5.4.

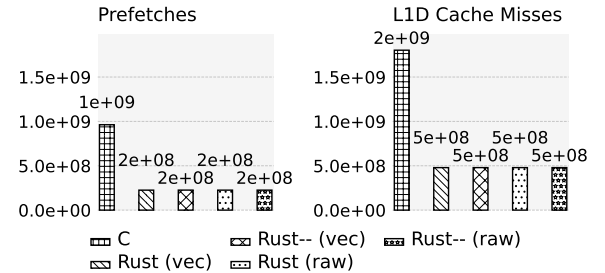


Figure 2: Hardware prefetches and L1D-cache misses for baseline C, baseline Rust, Rust--, and raw-pointer variants for Binary Search on AMD.

Higher Number of Prefetches in C. To analyze the differing cache behavior between C and Rust, we conducted a series of experiments with the Binary Search benchmark.

We first replaced the Rust vector with a raw pointer, which would ensure it would perform no bounds checks. As shown in the results in Figure 2, both the vector-based and raw-pointer based implementations have significantly lower L1D-cache miss rates. The only difference we could find between the C and Rust versions was that the C code was making $\sim 5\times$ as many prefetch requests as the Rust baseline. By disassembling the binary, we confirmed that these were hardware prefetcher requests, and because we could not control the BIOS settings on the CloudLab machines, we could not turn it off to further isolate the behavior. Furthermore, the hardware performance counter records only the L1D-cache misses resulting from *demand accesses* and not prefetches. Therefore, we hypothesize that the higher number of prefetches in C code evicts *hot* data in the cache, resulting in a higher L1D-cache miss rate.

5.4. Compiler Optimization Impact

As previously noted by Popescu et al. [63], the runtime safety checks can alter compiler optimizations in unpredictable ways. We discuss some of the interesting compiler optimization behaviors we observed during our analysis.

5.4.1. Promotion of Bounds Value to Registers.

We observed that for the Index-based Quicksort and the Selection Sort algorithms, despite performing additional memory-safety checks, as evidenced by the increase in dynamic branch instructions, there is little increase in the memory load instructions, with a 2% increase for Quicksort and no change for Selection Sort. This indicates that the compiler could successfully promote the vector size to a register, reducing any latency related to memory accesses. However, we did not observe this behavior in any of the other benchmarks, indicating that either some of the bound values could not be promoted to registers, or the extra register pressure from the bounds values caused other frequently-accessed variables to be spilled to the stack.

5.4.2. Converting Branch + Load Patterns to Conditional Moves. One possible compiler optimization is to turn conditional branch instructions followed by a memory operation—for example the `CMP rdi, rdx; JNE .LBB19_1; MOV rax, ptr` sequence shown in Figure 1, with a conditional move instruction, such as `CMP rdi, rdx; CMOVE rax, ptr; .` The CMOV instruction performs a memory move operation if the previous condition evaluates to a certain value. This reduces the number of executed instructions and essentially converts a control dependency, in the form of the branch instruction, into a data dependency.

Rust Mis-optimization to CMOV. As seen in Table 5, we observe this transformation happens across many of the benchmarks in Rust and typically with a greater number of CMOVs than in C. However, this optimization does not always lead to improved performance. As discussed in §5.3.1, the hardware branch predictor is very efficient for predictable branch checks, so this conversion likely is inefficient and is, in fact, a mis-optimization for benchmarks such as Binary Search. We suspect that the introduction

of conditional move instructions is why the Binary Search Rust benchmark performs 15% to 26% worse than C (for Intel and AMD, respectively), despite having 65% to 73% fewer L1D-cache misses and 94% fewer branch misses.

C Mis-optimization to CMOV. This mis-optimization, however, is not exclusive to Rust. In the SSSP benchmark, the C implementation is similarly mis-optimized using CMOV. When updating the distances array, which stores the shortest known distance from the source vertex to every other vertex, the C compiler computes the minimum using a CMOV instruction, whereas the Rust compiler emits a conditional branch followed by a memory store. While this difference leads to increased branches and therefore more branch mispredictions compared to C, it reduces the number of store instructions from over 1B in C to just over 200K in Rust, ultimately resulting in a 32% to 44% increase in performance for Rust.

Successful CMOV Optimization. However, not all *branch+load* $\rightarrow$ *CMOV* optimizations are misoptimizations and degrade performance. For the index-based variants of the Merge Sort benchmark, the C compiler applies the conditional load patterns to CMOV promotion, which reduces the overall number of branches, and *also the branch predictor misses*. However, the Rust compiler failed to apply this optimization to the Rust code, resulting in a significantly higher number of mispredictions ($\sim 18\times$ TDR, $\sim 20.5\times$ BUI). Our hypothesis is that there are some situations where Rust’s runtime checks prevent the application of the CMOV transformation. Table 5 shows that in the case of both indexed (non-iterator) implementations of Merge Sort, compiling with Rust-`enabled` the same CMOV optimization that C used and made a significant improvement in performance, even surpassing C’s performance for the bottom up (iterative) implementation.

Table 6 presents a summarizing table of all the benchmarks and their performance analysis. Together, these results highlight how Rust’s safety mechanisms influence low-level performance characteristics compared to C.

5.5. Aliasing Impact

Rust’s ownership model also affects how aggressively LLVM can optimize memory accesses. Compilers, such as LLVM [6] and GCC [1], use *alias analysis* (AA) [14, 73] to determine whether two pointers can refer to the same memory. When the compiler can prove they do not (NoAlias), it can better optimize the memory accesses by reordering or vectorizing them aggressively. LLVM conveys aliasing information through a combination of attributes, annotations, and metadata, which we collectively refer to as *alias information*. Rust’s borrow checker guarantees that no mutable reference coexists with any other reference to the same memory, and the compiler encodes this by emitting `noalias` alias information for `&mut T` and `&T` pointer parameters in the LLVM IR. C pointers carry no such alias information unless explicitly annotated with `restrict`. Given this, one would expect Rust to have more NoAlias as hypothesized by Villani et al. [82]. However, we

TABLE 6: Summary: Benchmark Variants and Rust’s Relative Performance to C at -O3

Benchmark	Variant	Overhead		Explanation
		AMD	Intel	
Binary Search	index-based	↑ 26%	↑ 15%	LLVM transforms predictable branches into slower CMOV, resulting in overall slowdown despite better cache & branch prediction.
Depth-First Search	index-based	↑ 5%	↑ 5%	Lower L1D and L2D misses offsets the increase in memory and branch instructions.
Breadth-First Search	index-based	↑ 10%	↑ 23%	Increase in dynamic memory instructions.
Single-Source Shortest Path (SSSP)	index-based	↓ 44%	↓ 22%	C uses CMOVs for each min computation and stores the result each time blindly. Rust only stores when update is needed.
	iterator	↓ 44%	↓ 23%	
Selection Sort	index-based	↑ 193%	↑ 57%	Increased instruction count due to bounds checks. Bad instruction layout in AMD causing uOC evictions and IC stalls
Quicksort	index-based	↑ 2%	↑ 7%	High branch predictor accuracy mitigates increased bounds checks with help from matching or better cache performance.
	iterator	↑ 71%	↑ 64%	Despite decent branch predictor performance, cache performance is poor. (L1D: 3.38x AMD - 48.32x Intel)
Merge Sort	index-based (TDR)	↑ 68%	↑ 58%	Bounds checks and unpredictable comparison lead to 18x branch misses on both architectures.
	iterator	↓ 15%	↓ 34%	Eliminates bounds checks and unpredictable branches leading to fewer mispredictions.
	index-based (BUI)	↑ 44%	↑ 43%	Same branch predictor performance as recursive version, but with better cache locality.
Matrix Multiplication	index-based	↑ 86%	↑ 71%	Increase in conditional branch instructions and dynamic load instructions due to triple nested loop.
	iterator	↓ 5%	↑ 1%	Eliminated bounds checks leads to same performance as C.
PageRank	index-based	↑ 40%	↑ 43%	Increased conditional branches and load instructions.
UTF-8 Decode (u8next)	index-based	↓ 33%	↓ 48%	Cache misses nearly eliminated (Intel) or greatly reduced while retaining similar instruction count to C.
CSV Validation (LibCSV)	index-based	↓ 13%	↓ 4%	Decrease in instruction count and improved cache performance.
Skip List	index-based	avg. ↓ 29.3%	avg. ↓ 6%	Lower L1D and L2D misses outweighs increase in dynamic memory instructions. High branch prediction accuracy mitigates increase in conditional branch instructions.

observe that this behavior is dependent on how the code is implemented and how memory is allocated.

Slice Type Improves Rust’s AA precision. To characterize the impact of Rust’s type system on aliasing information, we executed LLVM’s Alias Analysis [53] pass on the O3-optimized IR for PageRank and Merge Sort (Table 7). In PageRank, Rust achieves 41.1% NoAlias compared to 4.4% for C. This gap arises because the C function takes five raw pointers, of which three pointers are of `float*` type. The compiler cannot easily determine if pointers of the same type do not alias [32, 54] and these pointers remain MayAlias. The Rust version’s three immutable slice parameters all carry `noalias` alias information due to its stronger type system, achieving a 41.1% NoAlias.

Direct Allocator Metadata Improves C’s AA Precision. Merge Sort presents a different situation where C achieves 72.2% NoAlias compared to 56.6% for Rust. In this case, C’s merge function allocates arrays via `malloc`, whose return values carry *direct* alias information that propagates aggressively throughout the function. In the Rust implementation, the same arrays are allocated as `Vec<i32>` inside the function. Although Rust’s allocator calls also carry alias

information, it is conveyed through *scoped* metadata that propagates less effectively through LLVM’s optimization pipeline than the *direct* alias information on C’s `malloc` return value [41]. Additionally, Vec internals introduce additional pointer values, inflating the number of pairs that alias analysis must resolve, leaving Rust at a disadvantage.

Impact of Allocation Patterns on AA Precision. After observing this behavior, we implemented a modified version of Merge Sort in which the caller pre-allocates the temporary arrays and passes them as slice parameters into the merge function. This converts allocation-derived alias information into borrow-derived `noalias` on the function signature, raising the NoAlias rate to 70.0% for Rust—nearly matching C. In contrast, when the same modification is applied to C, NoAlias drops to just 6% and the total query count explodes from 36 to 465. This is because the pre-allocated arrays are now passed as raw pointers, stripping away the direct alias information that `malloc`’s return value had previously provided. Without it, LLVM must conservatively assume that every pointer pair may alias, leaving it unable to reorder or eliminate memory accesses.

	PageRank		Merge Sort		Merge Sort (Mod.)	
	C	Rust	C	Rust	C	Rust
Total Queries	406	253	36	120	465	120
NoAlias	4.4%	41.1%	72.2%	56.6%	6.0%	70.0%
MayAlias	95.5%	58.8%	27.7%	42.5%	93.9%	29.1%
MustAlias	0.0%	0.0%	0.0%	0.8%	0.0%	0.8%

TABLE 7: Aliasing results for PageRank and Merge Sort.

5.6. Intel vs. AMD Instruction-Mix Differences

As shown in Table 3 and Table 4, we observe notable differences in dynamic instruction-mix between the Intel and AMD architectures, particularly in store instructions. We used a Pin tool to further analyze the instruction mix and found that in BFS, the Intel platform exhibits a 47% increase in store instructions over AMD for the C implementation and a 60% increase for both Rust and Rust-. Merge Sort (TDR) shows a similar pattern, with increases of 16%, 38%, and 64% for C, Rust, and Rust- respectively. PageRank presents a more nuanced case: Intel shows an approximately 40% increase in stores for both C and Rust- but produces a nearly identical count for standard Rust. We attribute these differences primarily to architecture-specific optimizations within *glibc*, which uses runtime dispatching, via *ifunc*, to select architecture-tuned implementations of common functions such as *memset* and *memcpy*. For instance, the *glibc* implementation on Intel may favor repeated *STOSB* instructions, leveraging the Enhanced REP *MOVSB*/*STOSB* capability present on recent Intel processors, whereas the AMD-targeted variant may instead employ wider vectorized *MOV* instructions. Because a single vectorized store writes more data per instruction than a byte-granularity *STOSB*, the same operation can produce significantly different dynamic instruction count without altering the performance.

6. Discussion

FerroScope demonstrates that the performance of memory-safe systems languages is a dynamic interplay between the costs of the safety checks, the compiler-level optimizations, and microarchitectural speculation. Future language designs and transpilation systems can leverage this interplay to reconcile safety and performance without manual tuning. The cases where the performance of Rust code falls short reveal opportunities for further work.

Performance Modeling Challenges. Our results show that C-to-Rust transpilation overhead cannot be modeled as a uniform percentage increase. The Rust overhead varies substantially with workload characteristics such as data access and branch behavior. Traditional models that assume stable relationships between abstraction boundaries and microarchitectural events [20, 81] are therefore insufficient, and cannot accurately predict the performance cost. This necessitates the development of novel performance models that can accurately predict the performance of the transpiled Rust code from the C version.

Hardware Sensitivity and Generalizability. Our evaluation includes two widely deployed server-class microarchitectures—AMD EPYC and Intel Xeon Silver—which together represent the majority of modern datacenter and HPC deployments [56, 78]. Across both platforms, the key trends hold. Rust implementations that match or outperform C on one architecture generally do so on the other, and the microarchitectural mechanisms responsible are common to both vendors’ designs. This consistency suggests that our findings are likely to generalize to other modern processors.

Nonetheless, the magnitude of overhead varies between the two machines, confirming that microarchitectural features such as cache organization, branch predictor design, and out-of-order execution width influence the relative cost of runtime safety checks [58, 71]. Consequently, the performance ordering between competing implementation strategies may shift across hardware, undermining the notion of a single “best” translation.

Implications for Transpiler Toolchains. The lack of a definitive superior implementation strategy complicates the development of automated C-to-Rust transpiler toolchains. Present-day C-to-Rust transpilation toolchains [35, 36, 46] perform a line-by-line transpilation from C to Rust. Such systems would struggle to transpile C-style imperative code to idiomatic Rust paradigms. More recently, LLM-based transpiler toolchains [16, 37, 86] have proved modestly successful in transpiling C code to *idiomatic* Rust, but they struggle with correctness [39, 52, 86]. Eliciting performant output from these approaches compounds the challenge.

7. Related Works

Several studies have attempted to examined the performance gap between C and Rust. Popescu et al. [63] showed that developers struggle to identify costly bounds checks, and proposed NADER to selectively reintroduce them under a performance budget. Zhang et al. [88] found that Rust benchmarks, when implemented in a C-like manner, generally incur higher overhead than C, due to run-time checks and language constraints. Fromherz and Protzenko [42] observed that Rust can outperform unoptimized C but lags behind optimized C and that inlining causes significant gains, while removing bounds checks has little effect. Li et al. [51] reported inconsistent overheads for Rust in Linux drivers, citing coarse-grained locking and array bounds checks as key factors causing degradation, and also noted higher cache/TLB/branch miss rates due to Rust’s ownership patterns. Together, these studies highlight the need for a systematic performance analysis of Rust code translated from C.

8. Conclusion

FerroScope’s characterization shows that Rust’s safety checks are not the dominant source of overhead in transpiled code: compiler and microarchitectural effects often absorb their cost. This study contributes a systematic characterization of how microarchitectural and compiler effects give rise to Rust’s overheads.

References

- [1] “GNU Compiler Collection,” <https://gcc.gnu.org>.
- [2] “Libcsv: A CSV library in ANSI c.” [Online]. Available: <https://sourceforge.net/projects/libcsv/>
- [3] “Linux profiling with performance counters,” accessed: 2025-09-17. [Online]. Available: <https://perfwiki.github.io/main/>
- [4] “Rust — the linux kernel documentation,” <https://docs.kernel.org/rust/index.html>, accessed: 2025-10-07.
- [5] “Generic radix trees / sparse arrays — linux kernel documentation,” <https://www.kernel.org/doc/html/v5.4/core-api/generic-radix-tree.html>, 2018.
- [6] “The LLVM compiler infrastructure,” <https://llvm.org/>, 2020.
- [7] “Overview of the linux virtual file system (vfs): dentry & dcache,” <https://www.kernel.org/doc/html/v5.10/filesystems/vfs.html>, 2020.
- [8] “Pin 3.2 user guide,” <https://software.intel.com/sites/landingpage/pintool/docs/81205/Pin/html/>, 2020.
- [9] “Introducing rust code in firefox,” <https://firefox-source-docs.mozilla.org/build/buildsystem/rust.html>, 2025.
- [10] “Rust programming language,” <https://www.rust-lang.org/>, 2025.
- [11] “What is ownership?” <https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>, 2025.
- [12] “Cfs scheduler — the linux kernel documentation,” <https://docs.kernel.org/scheduler/sched-design-CFS.html>, –.
- [13] Allen Institute for AI, “Covid-19 open research dataset (cord-19),” <https://www.kaggle.com/datasets/allen-institute-for-ai/CORD-19-research-challenge>, 2020, accessed: 2025-09-28.
- [14] L. O. Andersen, “Program analysis and specialization for the c programming language,” Ph.D. dissertation, University of Copenhagen, 1994.
- [15] L. Backstrom, D. Huttenlocher, J. Kleinberg, and X. Lan, “Group formation in large social networks: membership, growth, and evolution,” in *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2006, pp. 44–54.
- [16] Y. Bai and T. Palit, “RustAssure: Differential symbolic testing for llm-transpiled c-to-rust code,” in *Proceedings of the 40th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2025.
- [17] S. Beamer, K. Asanović, and D. Patterson, “The gap benchmark suite,” *arXiv preprint arXiv:1508.03619*, 2015.
- [18] K. Boos, N. Liyanage, R. Ijaz, and L. Zhong, “Theseus: An experiment in operating system structure and state management,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 1–19. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/boos>
- [19] BurntSushi, “ripgrep: Line-oriented search tool in rust,” <https://github.com/BurntSushi/ripgrep>, accessed: 2025-10-03.
- [20] V. C. Cabezas and M. Püschel, “Extending the roofline model: Bottleneck analysis with microarchitectural constraints,” in *2014 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2014, pp. 222–231.
- [21] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, “cudnn: Efficient primitives for deep learning,” *arXiv preprint arXiv:1410.0759*, 2014.
- [22] U. Consortium, “NormalizationTest.txt,” <https://www.unicode.org/Public/UNIDATA/NormalizationTest.txt>, 2025, unicode Character Database - Normalization Forms Test File.
- [23] G. Contributors, “TreeRegion scheduling in gcc,” p. –.
- [24] G. P. . contributors, “Gcc internals: Control and data flow support,” <https://www.airs.com/dnovillo/200711-GCC-Internals/200711-GCC-Internals-4-cfg-cg-df.pdf>, 2007.
- [25] J. Corbet, “Rust in the linux kernel,” *LWN.net*, Dec 2022, coverage of Rust support merged into Linux for writing drivers. [Online]. Available: <https://lwn.net/Articles/915044/>
- [26] Crates.io, “e1000-driver crate,” <https://crates.io/crates/e1000-driver>, 2023, rust crate for Intel e1000 NICs; docs at docs.rs/e1000-driver.
- [27] C. Cui, “Finding performance issues in rust projects,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 2423–2425.
- [28] A. C. De Melo, “The new linux ‘perf’ tools,” in *Slides from Linux Kongress*, vol. 18, no. 1, 2010.
- [29] De Vries, Folkert, “zlib-rs is faster than c,” <https://www.overleaf.com/project/689a561bfedd8929d9a5870b>, accessed: 2025-09-20.
- [30] R. Dentato, “u8c: Fast validating utf-8 encoder/decoder for c,” GitHub repository, 2025, accessed: 2025-09-21. [Online]. Available: <https://github.com/rdentato/u8c>
- [31] N. Developers, “Rust binding for nginx,” <https://github.com/nginx/nginx-rust>, 2007.
- [32] A. Diwan, K. S. McKinley, and J. E. B. Moss, “Type-based alias analysis,” in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 1998, pp. 106–117.
- [33] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra, “The design and operation of CloudLab,” in *Proceedings of the USENIX Annual Technical Conference (ATC)*, Jul. 2019, pp. 1–14. [Online]. Available: <https://www.flux.utah.edu/paper/duplyakin-atc19>
- [34] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf, “Aliasing limits on translating c to safe rust,” vol. 7, pp. 551–579, publisher: ACM New York, NY, USA.
- [35] —, “Aliasing limits on translating c to safe rust,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 551–579, 2023.
- [36] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, “Translating c to safer rust,” vol. 5, pp. 1–29, publisher: ACM New York, NY, USA.
- [37] H. F. Eniser, H. Zhang, C. David, M. Wang, M. Christakis, B. Paulsen, J. Dodds, and D. Kroening, “Towards translating real-world code with LLMs: A study of translating to rust.”
- [38] I. M. Esmer *et al.*, “New 3rd Gen Intel Xeon Scalable Processor (Codename: Ice Lake-SP),” in *IEEE Hot Chips 32 Symposium (HCS)*, Virtual, Aug. 2020. [Online]. Available: https://hc32.hotchips.org/assets/program/conference/day1/HotChips2020_Server_Processors_Intel_Irma_ICX-CPU-final3.pdf
- [39] M. Farrukh, S. Shah, B. Coskun, and M. Polychronakis, “Safe-trans: Llm-assisted transpilation from c to rust,” *arXiv preprint arXiv:2505.10708*, 2025.
- [40] A. Fedorova, S. Blagodurov, and S. Zhuravlev, “Managing contention for shared resources on multicore processors,” *Communications of the ACM*, vol. 53, no. 2, pp. 49–57, 2010.
- [41] H. Finkel, “Upcoming changes/additions to scoped-noalias metadata,” LLVM Developers Mailing List, https://groups.google.com/g/llvm-dev/c/S_fZzVK9UIs, 2015, accessed: 2025-09-28.
- [42] A. Fromherz and J. Protzenko, “Compiling c to safe rust, formalized,” *arXiv preprint arXiv:2412.15042*, 2024.
- [43] R. Gamble, “libcsv: Fast and flexible csv library written in pure ansi c that can read and write csv data.” accessed: 2025-09-18. [Online]. Available: <https://github.com/rgamble/libcsv>
- [44] B. Gao, Q. Kang, H.-W. Tee, K. T. N. Chu, A. Sanaee, and D. Jevdjic, “Scalable and effective page-table and {TLB} management on {NUMA} systems,” in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 2024, pp. 445–461.
- [45] Google, “google/leveldb: A fast key/value storage library,” <https://github.com/google/leveldb>, accessed: 2025-09-27.

- [46] C. D. Hanliang Zhang, Y. Yu, and M. Wang, “Ownership guided c to rust translation.”
- [47] Intel Corporation. (2021) Intel Xeon Silver 4314 Processor (24M Cache, 2.40 GHz) — Product Specifications. [Online]. Available: <https://ark.intel.com/content/www/us/en/ark/products/215269/intel-xeon-silver-4314-processor-24m-cache-2-40-ghz.html>
- [48] R. Landley, “Red-black trees (rbtree) in linux,” <https://www.kernel.org/doc/html/v5.9/core-api/rbtree.html>, 2007.
- [49] M. Larabel. (2025, May) Rust coreutils 0.1 released with big performance gains - can match or exceed gnu speed. Accessed: 2026-04-30. [Online]. Available: <https://www.phoronix.com/news/Rust-Coreutils-0.1-Released>
- [50] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney, “Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters,” *Internet Mathematics*, vol. 6, no. 1, pp. 29–123, 2009.
- [51] H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu, “An empirical study of {Rust-for-Linux}: The success, dissatisfaction, and compromise,” in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 2024, pp. 425–443.
- [52] R. Li, B. Wang, T. Li, P. Saxena, and A. Kundu, “Translating c to rust: Lessons from a user study,” *arXiv preprint arXiv:2411.14174*, 2024.
- [53] LLVM Project, “LLVM alias analysis infrastructure — the -aa-eval-pass,” <https://llvm.org/docs/AliasAnalysis.html#the-aa-eval-pass>, accessed: 2026-05-19.
- [54] —, “LLVM Type Based Alias Analysis,” <https://llvm.org/docs/LangRef.html#tbaa-metadata>, accessed: 2026-05-19.
- [55] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, “Pin: building customized program analysis tools with dynamic instrumentation,” in *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation*, 2005.
- [56] D. McCarron. (2025) Mercury Research Q2 2025 x86 Market Share Report. As reported by Network World and Tom’s Hardware. [Online]. Available: <https://www.networkworld.com/article/4040551/despise-the-hubhub-intel-is-holding-onto-server-market-share.html>
- [57] I. Mihajlovic, “Rust vs c++: Competition or evolution in systems programming for 2026,” <https://blog.jetbrains.com/rust/2025/12/16/rust-vs-cpp-comparison-for-2026/>, Dec. 2025, JetBrains RustRover Blog.
- [58] S. Nagarakatte, M. M. Martin, and S. Zdancewic, “Hardware-enforced comprehensive memory safety,” *IEEE Micro*, vol. 33, no. 3, pp. 38–47, 2013.
- [59] V. Narayanan, T. Huang, D. Detweiler, D. Appel, Z. Li, G. Zellweger, and A. Burtsev, “Redleaf: Isolation and communication in a safe operating system,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 21–39. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/narayanan-vikram>
- [60] V. Nitin, R. Krishna, L. L. d. Valle, and B. Ray, “C2saferust: Transforming c projects into safer rust with neurosymbolic techniques,” *arXiv preprint arXiv:2501.14257*, 2025.
- [61] “Java platform, standard edition hotspot virtual machine garbage collection tuning guide,” <https://docs.oracle.com/en/java/javase/21/gctuning/index.html>, Oracle, 2024.
- [62] L. Page, S. Brin, R. Motwani, and T. Winograd, “The pagerank citation ranking: Bringing order to the web.” Stanford infolab, Tech. Rep., 1999.
- [63] N. Popescu, Z. Xu, S. Apostolakis, D. I. August, and A. Levy, “Safer at any speed: automatic context-aware safety enhancement for rust,” *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–23, 2021.
- [64] K. Purdy, “Linux leaders pave a path for rust in kernel, while supporting c veterans,” *Ars Technica*, Feb. 2025, accessed: 2025-10-07.
- [65] Redis, “Redis,” <https://redis.io/>, 2019.
- [66] Redox OS Project, “Redox os,” <https://www.redox-os.org/>, 2024.
- [67] S. Rudra, “Rust coreutils are performing worse than gnu coreutils in ubuntu,” <https://news.itsfoss.com/ubuntu-uutils-performance-issues>, accessed: 2025-09-20.
- [68] J. H. Ryoo, N. Guler, S. Song, and L. K. John, “Rethinking tlb designs in virtualized environments: A very large part-of-memory tlb,” *ACM SIGARCH Computer Architecture News*, vol. 45, no. 2, pp. 469–480, 2017.
- [69] serde-rs, “serde: Serialization framework for rust,” <https://github.com/serde-rs/serde>, accessed: 2025-10-03.
- [70] J. Shun and G. E. Blelloch, “Ligra: a lightweight graph processing framework for shared memory,” in *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*, 2013, pp. 135–146.
- [71] K. Skadron, P. S. Ahuja, M. Martonosi, and D. W. Clark, “Branch prediction, instruction-window size, and cache size: Performance trade-offs and simulation techniques,” *IEEE Transactions on Computers*, vol. 48, no. 11, pp. 1260–1281, 2002.
- [72] *B-Tree Pages and Binary Search in SQLite*, <https://www.sqlite.org/src/file/src/btree.c>, SQLite Documentation, 2024.
- [73] B. Steensgaard, “Points-to analysis in almost linear time,” in *Proceedings of the 23rd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1996, pp. 32–41.
- [74] F. D. E. Team, “facebook/rocksdb: A library that provides an embeddable, persistent key-value store,” <https://github.com/facebook/rocksdb>, accessed: 2025-09-27.
- [75] The Go Team, “A guide to the go garbage collector,” <https://go.dev/doc/gc-guide>, 2023.
- [76] The Linux man-pages project, *perf_event_open(2)*, man7.org.
- [77] Tock Project, “Tock os: A secure embedded operating system for microcontrollers,” <https://www.tockos.org>, 2024, kernel and drivers written in Rust.
- [78] TOP500. (2024) TOP500 Highlights – November 2024. [Online]. Available: <https://www.top500.org/lists/top500/2024/11/highs/>
- [79] Uber AI × Uber Maps, “Deepeta: How uber predicts arrival times,” <https://www.uber.com/blog/deepeta-how-uber-predicts-arrival-times/>, 2022, accessed: 2025-09-27.
- [80] V. S. P. K. Vadrevu, L. Xing, and W. G. Aref, “What cannot be skipped about the skiplist: A survey of skiplists and their applications in big data systems,” *arXiv preprint arXiv:2403.04582*, 2024.
- [81] S. Van den Steen, S. Eyerman, S. De Pestel, M. Mechri, T. E. Carlson, D. Black-Schaffer, E. Hagersten, and L. Eeckhout, “Analytical processor performance and power modeling using micro-architecture independent characteristics,” *IEEE Transactions on Computers*, vol. 65, no. 12, pp. 3537–3551, 2016.
- [82] N. Villani, J. Hostert, D. Dreyer, and R. Jung, “Tree borrows,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, pp. 1–24, Jun. 2025. [Online]. Available: <https://doi.org/10.1145/3735592>
- [83] J. Wallen, “Initial support for rust in the linux kernel is finally released,” *Linux Magazine*, Dec. 2022, accessed: 2025-10-07.
- [84] L. L. Wang, K. Lo, Y. Chandrasekhar, R. Reas, J. Yang, D. Burdick, D. Eide, K. Funk, Y. Katsis, R. M. Kinney, Y. Li, Z. Liu, W. Merrill, P. Mooney, D. A. Murdick, D. Rishi, J. Sheehan, Z. Shen, B. Stilson, A. D. Wade, K. Wang, N. X. R. Wang, C. Wilhelm, B. Xie, D. M. Raymond, D. S. Weld, O. Etzioni, and S. Kohlmeier, “CORD-19: The COVID-19 open research dataset,” in *Proceedings of the 1st Workshop on NLP for COVID-19 at ACL 2020*, K. Verspoor, K. B. Cohen, M. Dredze, E. Ferrara, J. May, R. Munro, C. Paris, and B. Wallace, Eds. Online: Association for Computational Linguistics, Jul. 2020. [Online]. Available: <https://aclanthology.org/2020.nlp-covid19-acl-1/>

- [85] M. Wells, "Rust vs c++: Which should you learn in 2026?" <https://rustify.rs/articles/rust-vs-cpp-2026>, Feb. 2026, updated April 2026.
- [86] A. Z. Yang, Y. Takashima, B. Paulsen, J. Dodds, and D. Kroening, "VERT: Verified equivalent rust transpilation with large language models as few-shot learners."
- [87] H. Zhang, C. David, Y. Yu, and M. Wang, "Ownership guided c to rust translation," in *International Conference on Computer Aided Verification*. Springer, 2023, pp. 459–482.
- [88] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu, "Towards understanding the runtime performance of rust," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–6.
- [89] T. Zhou, H. Lin, S. Jha, M. Christodorescu, K. Levchenko, and V. Chandrasekaran, "LLM-driven multi-step translation from c to rust using static analysis."